7zpus.swe@gmail.com

Specifica Tecnica

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026/02/28	Laoud Zakaria	Georgescu Diana	Creazione del documento e stesura struttura iniziale

Indice

1 Introduzione **5**
1.1 Obiettivo del documento 5
1.2 Glossario 5
1.3 Riferimenti 5
1.3.1 Riferimenti normativi 5
1.3.2 Riferimenti informativi 5

2 Tecnologie utilizzate **7**

3 Linguaggio di Programmazione: TypeScript **7**
3.1 Deployment: Electron 7
3.2 Framework Frontend: Angular 8
3.3 Librerie Aggiuntive 9
3.4 Tecnologie di analisi statica 10
3.5 Tecnologie di analisi dinamica 10

4 Contesto del sistema **10**
4.1 Modello architetturale 11
4.2 Descrizione degli attori esterni 11

5 Architettura dei container **12**
5.1 Renderer 13
5.1.1 Struttura architetturale 13
5.2 [Main] 13
5.2.1 Tecnologia: Node.js 13

6 Architettura delle componenti **14**
6.1 [Componente 1] 14
6.1.1 Descrizione 14
6.1.2 Interfacce 14
6.1.3 Dipendenze 14
6.1.4 Tracciamento dei requisiti 14
6.2 [Componente 2] 14
6.2.1 Descrizione 14
6.2.2 Interfacce 14
6.2.3 Dipendenze 14
6.2.4 Tracciamento dei requisiti 14
6.3 Componenti di Ricerca (FrontEnd) 15
6.3.1 Descrizione 15

7 Backend - Search **15**
7.1 Panoramica 15
7.2 Componenti 16
7.2.1 SearchIpcAdapter 18

7.2.2 Use Case di ricerca 19
7.2.3 Modelli di dominio per la ricerca 20
7.2.4 Mapper di ricerca 22
7.2.5 Repository e DAO di ricerca 24
7.2.6 DocumentMetadataQueryBuilder 25

Elenco delle figure

1 Diagramma C1 - Contesto del sistema *DIPReader* 11
2 Diagramma C2 - Container del sistema *DIPReader* 12
3 Diagramma C3 – Componenti del modulo di ricerca 16
4 Diagramma C3 – Componenti del modulo di ricerca (Repository e DAO) . 17
5 Diagramma delle classi – Search: IPC Adapter e Use Cases 18
6 Diagramma delle classi – Search: Query Model e Mapper 20
7 Diagramma delle classi – Search: Repository e DAO 24

Elenco delle tabelle

2 Dettagli sulla scelta di TypeScript come linguaggio di programmazione . 7
4 Dettagli sulla scelta di Electron come piattaforma di deployment 7
6 Dettagli sulla scelta di Angular come framework frontend 8
8 Dettagli sulle scelte di librerie aggiuntive 9
10 Dettagli sulle scelte di tecnologie di analisi statica 10
12 Dettagli sulle scelte di tecnologie di analisi dinamica 10

1 Introduzione

1.1 Obiettivo del documento

L'obiettivo del presente documento è descrivere in modo rigoroso le scelte tecnologiche e l'architettura del prodotto *DIPReader*, adottando il modello di progettazione *C4*.
L'analisi verrà condotta partendo dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container, delle componenti e, infine, del codice, al fine di fornire una rappresentazione completa, coerente e sistematica del sistema.

1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'Ingegneria del Software, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario (v2.0.0).

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Norme di Progetto v2.0.0 (v2.0.0)
- Standard SDD/IEEE/1016:2009(ultimo accesso: 02/03/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 02/03/2026)

1.3.2 Riferimenti informativi

- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Dispense su Dependency Injection (Ultimo accesso: 02/03/2026)
- Dispense su OOP (Ultimo accesso: 02/03/2026)
- Dispense su Diagrammi delle classi (Ultimo accesso: 02/03/2026)
- Dispense su Pattern architetturali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern creazionali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern strutturali (Ultimo accesso: 02/03/2026)
- Dispense su Principi SOLID (Ultimo accesso: 02/03/2026)
- Dispense sulla Progettazione software (Ultimo accesso: 02/03/2026)
- Dispense sulla Qualità del software (Ultimo accesso: 02/03/2026)
- Repository Architettura esagonale (Kotlin) (Ultimo accesso: 02/03/2026)

- Repository Architettura esagonale (Java) (Ultimo accesso: 02/03/2026)
- Repository Ingegneria del software - swe-imdb (Ultimo accesso: 02/03/2026)
- C4 Model (Ultimo accesso: 03/03/2026)
- PlantUML (Ultimo accesso: 03/03/2026)
- Riferimenti a scelte tecnologiche:
  - TypeScript (Ultimo accesso: 03/03/2026)
  - Angular (Ultimo accesso: 03/03/2026)
  - Electron (Ultimo accesso: 03/03/2026)
  - SQLite (Ultimo accesso: 03/03/2026)
  - sqlite-vss (Ultimo accesso: 03/03/2026)
  - ONNX Runtime (Ultimo accesso: 03/03/2026)
  - Transformers.js (Ultimo accesso: 03/03/2026)
  - Node.js (Ultimo accesso: 03/03/2026)
- Glossario (v2.0.0)
- Piano di Progetto (v2.0.0)
- Piano di Qualifica (v2.0.0)
- Verbali interni:
- Verbali esterni:

2 Tecnologie utilizzate

La scelta delle teccnologie è stata guidata dalla necessità di rispetta i vincoli richiesti dalla proponente legati ad autoconsistenza, portabilità e facilità d'uso, nonché da considerazioni relative alla facilità di sviluppo, alla disponibilità di risorse e alla compatibilità con i requisiti funzionali e non funzionali del sistema.

3 Linguaggio di Programmazione: TypeScript

Scelta	Versione	Motivazione
TypeScript	5.9.2	Scelto per la sua capacità di fornire un sistema di tipi statico che migliora la sicurezza del codice e facilita la manutenzione, riducendo il rischio di errori comuni durante lo sviluppo. Inoltre, TypeScript è ampiamente supportato da un ecosistema ricco di librerie e strumenti, e si integra perfettamente con tecnologie come Angular ed Electron.

Tabella 2: Dettagli sulla scelta di TypeScript come linguaggio di programmazione

3.1 Deployment: Electron

Scelta	Versione	Motivazione
Electron	30.0.0	Scelto per la sua capacità di consentire lo sviluppo di applicazioni desktop multipiattaforma utilizzando tecnologie web, garantendo così la portabilità del prodotto su diversi sistemi operativi. Electron permette di creare un'esperienza utente nativa, sfruttando le potenzialità di Node.js per l'accesso al filesystem e alle risorse locali, facilitando l'integrazione con librerie e strumenti di terze parti.

Tabella 4: Dettagli sulla scelta di Electron come piattaforma di deployment

3.2 Framework Frontend: Angular

Scelta	Versione	Motivazione
Angular	21.2.5	Scelto per la sua capacità di supportare lo sviluppo di applicazioni web complesse e scalabili, con un'architettura basata su componenti che favorisce la modularità e la manutenibilità del codice. Inoltre, Angular offre un'ampia gamma di strumenti e funzionalità integrate, come la gestione dello stato, il routing e il supporto per la creazione di interfacce utente reattive, che lo rendono adatto a soddisfare i requisiti funzionali del progetto. La sua implementazione della Dependency Injection facilita la gestione delle dipendenze tra i componenti, migliorando la testabilità e la manutenibilità del codice.

Tabella 6: Dettagli sulla scelta di Angular come framework frontend

3.3 Librerie Aggiuntive

Scelta	Versione	Motivazione
Better SQLite	12.8.0	SQLite è un database relazionale che non richiede un server separato e può essere integrato direttamente nell'applicazione, facilitando lo sviluppo e il deployment. La sua leggerezza e portabilità lo rendono adatto a un'ampia gamma di piattaforme, garantendo la compatibilità con i requisiti di portabilità del progetto.
sqlite-vss	0.1.2	Scelto per la sua capacità di fornire una versione virtuale di SQLite, che consente di eseguire query su dati in memoria senza la necessità di creare un database fisico dei vettori per la ricerca semantica, semplificando così l'integrazione con il sistema.
ONNX Runtime	1.17.3	Scelto per la sua capacità di eseguire modelli di machine learning in modo efficiente e flessibile, supportando una vasta gamma di modelli e framework, facilitando l'integrazione di funzionalità di intelligenza artificiale nel prodotto.
Transformers.js	3.8.1	Scelto per la sua capacità di fornire un'interfaccia JavaScript per l'utilizzo di modelli di trasformatori, facilitando l'integrazione di funzionalità avanzate di elaborazione del linguaggio naturale e machine learning nel prodotto.

Tabella 8: Dettagli sulle scelte di librerie aggiuntive

3.4 Tecnologie di analisi statica

Scelta	Versione	Motivazione
SonarQube	2026.2	Scelto per la sua capacità di identificare e segnalare pattern di codice problematici, migliorando la qualità del codice e facilitando la manutenzione del progetto. Integrato nella pipeline di sviluppo, SonarQube consente di monitorare costantemente la qualità del codice, identificando potenziali bug, vulnerabilità e problemi di manutenzione, contribuendo così a garantire un prodotto finale robusto e affidabile.
Prettier	2.9.3	Scelto per la sua capacità di formattare automaticamente il codice in modo coerente, migliorando la leggibilità e facilitando la collaborazione tra i membri del team. Prettier supporta una vasta gamma di linguaggi e può essere facilmente integrato con editor di codice e strumenti di sviluppo, semplificando il processo di mantenimento di uno stile di codifica uniforme.

Tabella 10: Dettagli sulle scelte di tecnologie di analisi statica

3.5 Tecnologie di analisi dinamica

Scelta	Versione	Motivazione
Vitest	2.3.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni JavaScript e per il suo controllo dei componenti di Mock.
Playwright	1.59.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni web, con possibilità di testare flussi completi di interazione utente. Utilizzato quindi per i test E2E.

Tabella 12: Dettagli sulle scelte di tecnologie di analisi dinamica

4 Contesto del sistema

I vincoli di progetto, come specificato nell'Analisi dei Requisiti, richiedono che il sistema sia portabile, facile da usare e autoconsistente. Il caso d'uso principale del sistema è quello di un utente che desidera esplorare e interrogare un DIP per estrarre documenti fiscali e non, potendone verificare la validità e autenticità.

4.1 Modello architetturale

I vincoli di progetto hanno guidato la progettazione del sistema, influenzando le scelte tecnologiche e architetturali. Il sistema progettato quindi è pensato per non utilizzare tecnologie che richiedano installazioni o configurazioni complesse da parte dell'utente. I servizi offerti sono tutti accessibili in assenza di connettività, complesa la ricerca semantica.
Gli unici attori esterni coinvolti sono il file *system locale* da cui viene letto il DIP. STAMPA DA VEDERE SE HA SENSO METTERLO

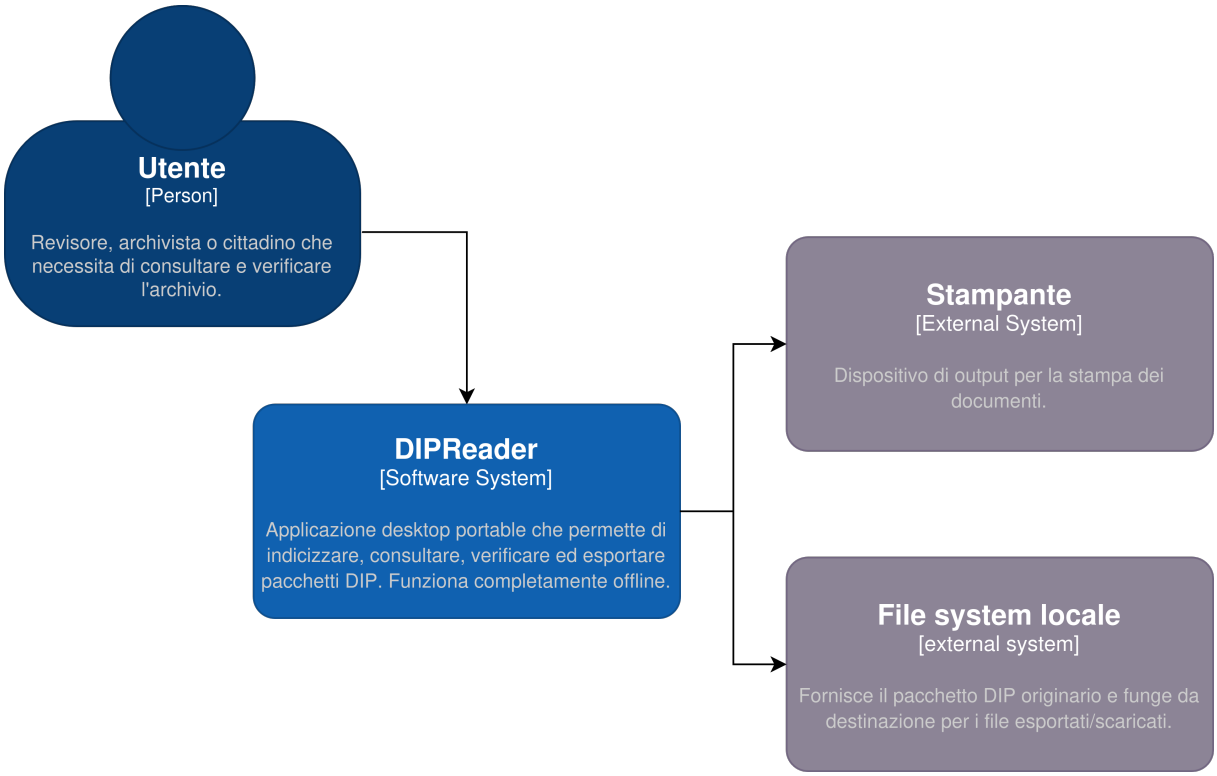

Figura 1: Diagramma C1 - Contesto del sistema *DIPReader*

4.2 Descrizione degli attori esterni

Gli unici attori esterni come già detto sono:

- il file system locale da cui viene letto il DIP
- STAMPA DA VEDERE SE HA SENSO METTERLO

5 Architettura dei container

La progettazione del sistema *DIPReader* è stata guidata dai principi di modularità, scalabilità e manutenibilità, con l'obiettivo di creare un'architettura che possa evolversi nel tempo e adattarsi a nuove esigenze e normative AGID.
In particolare, è stata completamente scorporata la logica di business dalla logica di presentazione, esponendo come unico punto di accesso comune gli *IPC channel*.
Il sistema così facendo permette di avere un backend facilmente estendibile, modificabile e soprattutto che si adatta allla struttura del DIP autonomamente. D'altra parte il frontend è completamente disaccoppiato ma strutturato in modo che i componenti di presentazione siano facilmente sostituibili, modificabili e soprattutto adattabili a nuove esigenze.

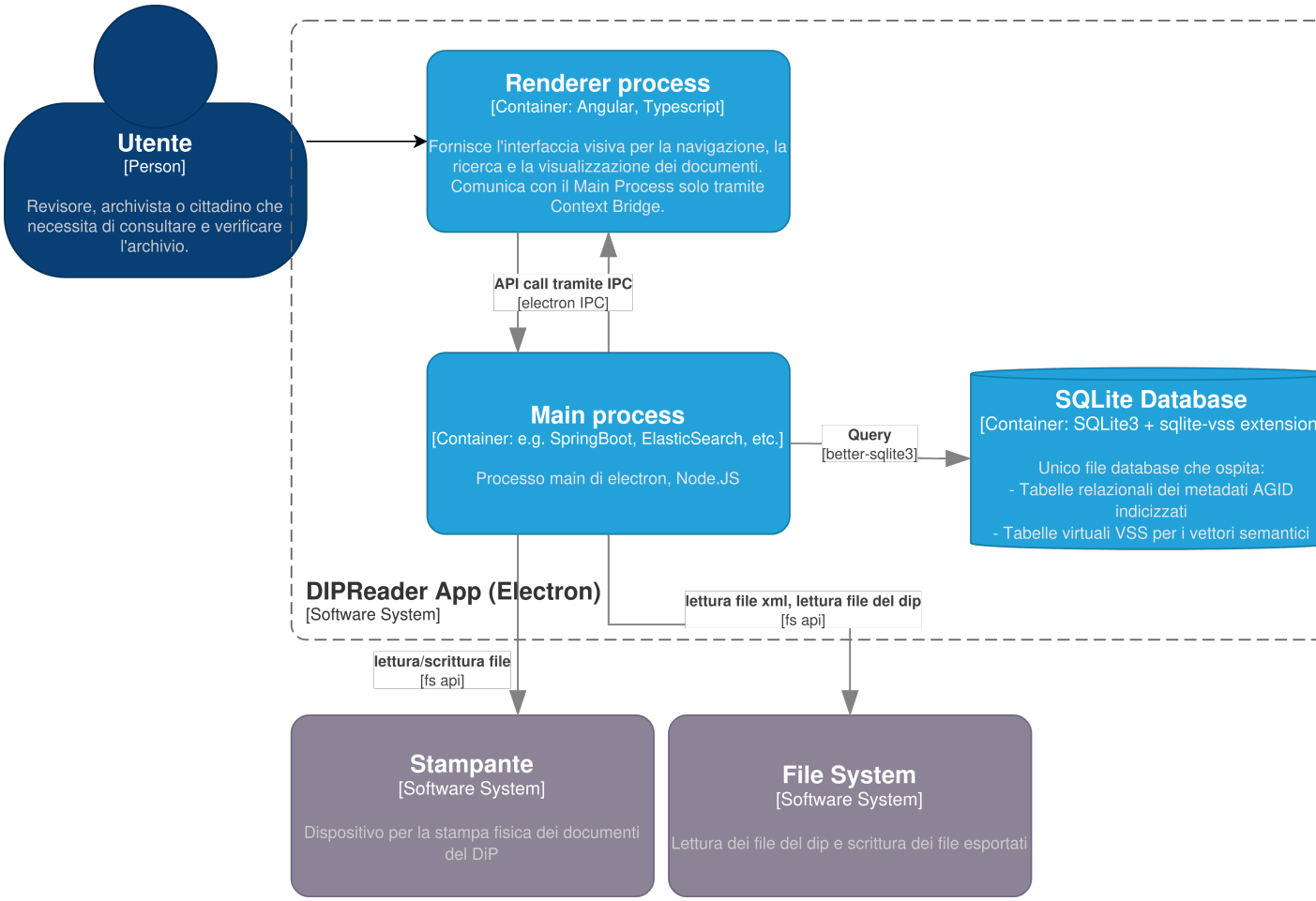

Figura 2: Diagramma C2 - Container del sistema *DIPReader*

5.1 Renderer

5.1.1 Struttura architetturale

Il container *Renderer* è responsabile della presentazione dell'interfaccia utente e dell'interazione con l'utente. Per la gestione quindi dell'interfaccia è stato scelto di strutturare il container per componenti. Ogni componente a sua volta è strutturato da una serie di services che si occupano di gestire la logica di presentazione, e da una serie di componenti di presentazione che si occupano di mostrare i dati all'utente.
Visto poi che le interazioni tra i diversi componenti di presentazione sono limitate, è stato scelto di strutturare il container in modo che ogni componente sia completamente disaccoppiato dagli altri. Gli unici punti di contatto risultano infatti essere il sistema di routing e i canali IPC, che permettono ai componenti di comunicare tra loro e con il backend.
In presenza poi di componenti come per esempio l'esportazione dei file, che richiedono di essere inseriti in più punti del sistema, è stato scelto di strutturare il container in modo che questi componenti siano facilmente integrabili in qualsiasi punto del sistema, senza dover modificare la struttura del container o dei componenti di presentazione grazie a delle semplici Injection di dipendenze.
È poi stato applicato il pattern Smart and Dumb Components, che permette di separare la logica di presentazione pura dalla logica di business che la coordina.

5.2 [Main]

5.2.1 Tecnologia: Node.js

Descrizione

6 Architettura delle componenti

6.1 [Componente 1]

6.1.1 Descrizione

6.1.2 Interfacce

6.1.3 Dipendenze

6.1.4 Tracciamento dei requisiti

6.2 [Componente 2]

6.2.1 Descrizione

6.2.2 Interfacce

6.2.3 Dipendenze

6.2.4 Tracciamento dei requisiti

6.3 Componenti di Ricerca (FrontEnd)

6.3.1 Descrizione

I componenti di ricerca sono responsabili della gestione delle funzionalità di compilazione dei campi di ricerca.
Come già anticipato il pattern applicato è quello di Smart and Dumb Components, per cui i componenti di ricerca sono suddivisi in componenti di pura presentazione e gestiti da i componenti "*Padri*", che si occupano di gestire la logica di questi ultimi nonchè le interazioni con altri servizi come la validazione dei filtri, la gestione dei campi di ricerca dinamici e il routing della sezione dei risultati. Nel contesto della ricerca questo coomponente Padre è SearchPageComponent.
Come si può notare questo componente sfrutta la dependency injection nativa di Angular per risolvere le dipendenze. Inoltre sono iniettate le interfaccie dei servizi necessari per il funzionamento e non gli effettivi servizi, applicando il principio di sostituzione di Liskov.

7 Backend - Search

7.1 Panoramica

Il modulo di ricerca espone al frontend, tramite IPC (Inter-Process Communication) di Electron, le funzionalità di interrogazione del pacchetto DiP indicizzato. Il punto di ingresso è *SearchIpcAdapter*, che registra su *IpcMain* i canali di comunicazione e delega l'elaborazione ai rispettivi Use Case.
Il modulo supporta quattro modalità di ricerca principali:

- **Ricerca per classe documentale**: ricerca le *DocumentClass* per nome tramite *SearchDocumentalClassUC*.
- **Ricerca per processo**: ricerca i *Process* per UUID tramite *SearchProcessUC*.
- **Ricerca strutturata per metadati**: ricerca i *Document* applicando un albero di filtri logici tramite *SearchDocumentsUC*, costruito a partire da un *SearchRequestDTO* validato con Zod e trasformato in un *SearchDocumentsQuery* tramite *MetadataKeyMapper*.
- **Ricerca semantica**: ricerca i *Document* per similarità vettoriale tramite *SearchSemanticUC*, convertendo la query testuale in un vettore di embedding tramite *IWordEmbedding* e calcolando la *cosine similarity* rispetto ai vettori indicizzati.

Sono inoltre esposti due canali ausiliari: uno per recuperare le chiavi di metadati personalizzati presenti nel pacchetto (*GetCustomMetadataKeysUC*), e uno per ottenere lo stato del motore AI (modello inizializzato, numero di documenti indicizzati).
I risultati vengono mappati in DTO di presentazione (*DocumentSearchResult*, *ProcessSearchResult*, *DocumentClassSearchResult*) prima di essere restituiti al frontend, mantenendo il dominio disaccoppiato dal livello di trasporto IPC.

7.2 Componenti

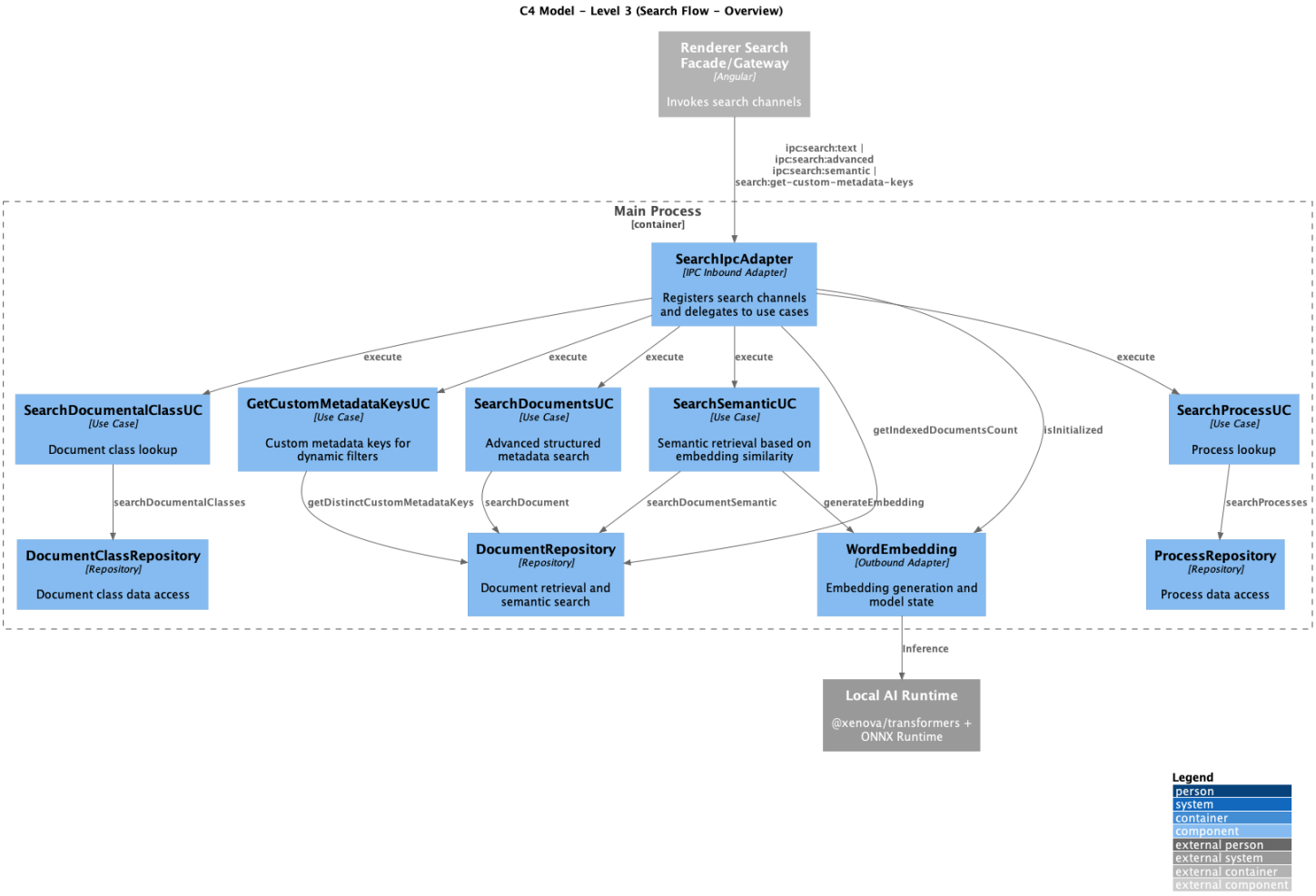

Figura 3: Diagramma C3 – Componenti del modulo di ricerca

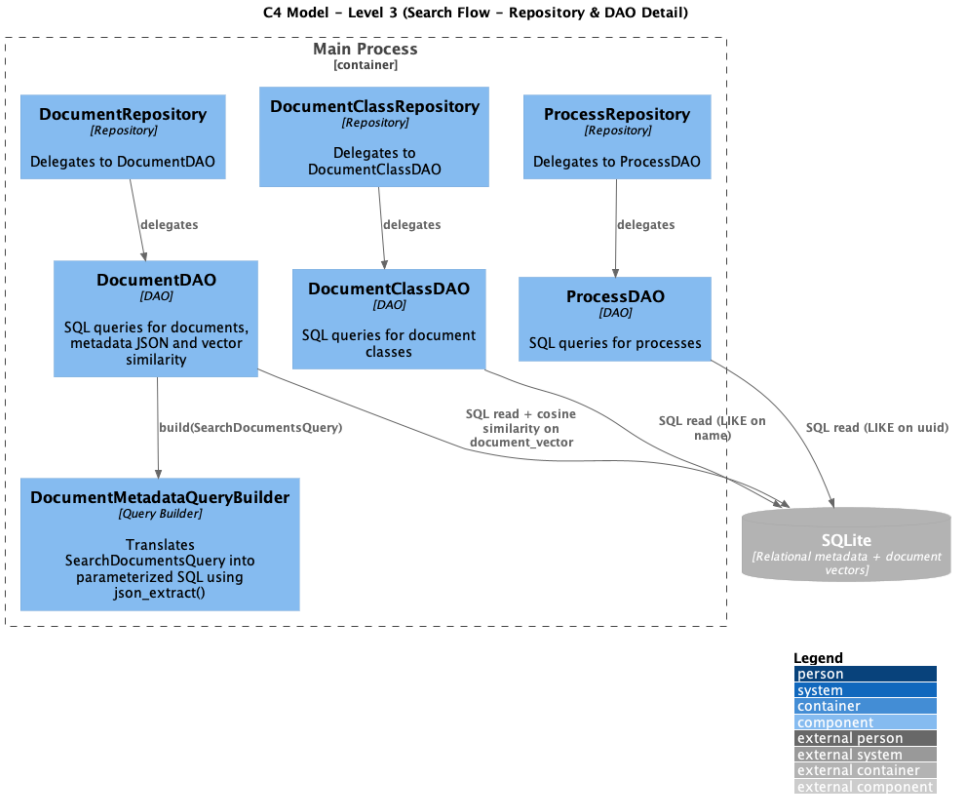

Figura 4: Diagramma C3 – Componenti del modulo di ricerca (Repository e DAO)

7.2.1 SearchIpcAdapter

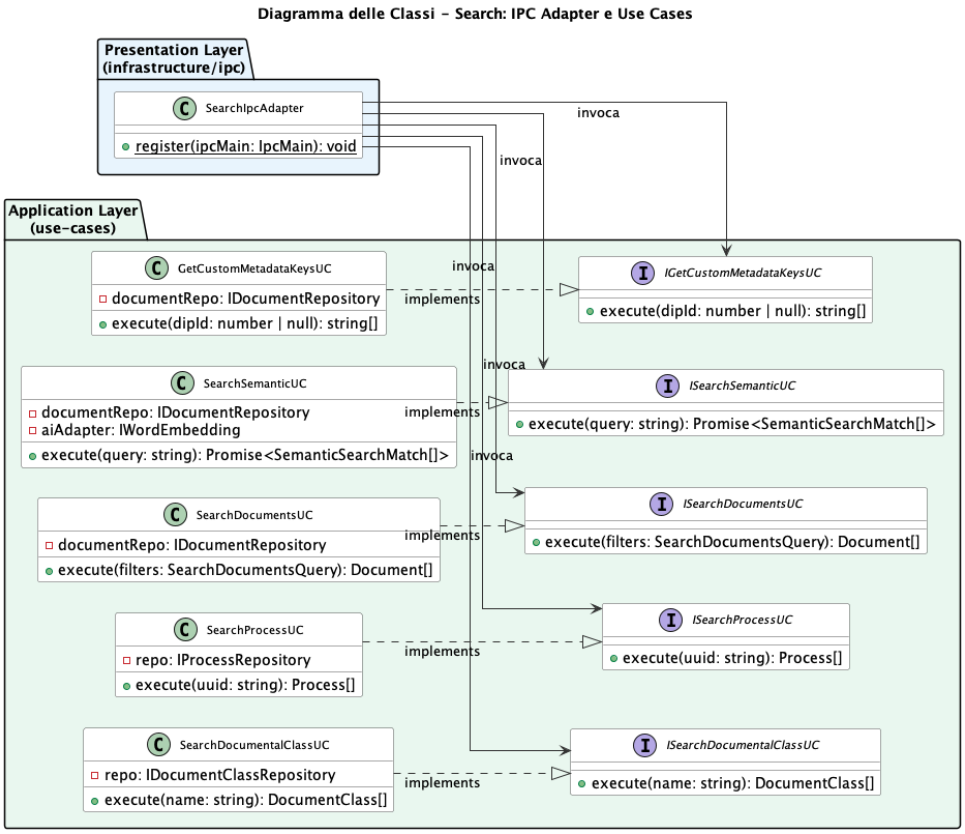

Figura 5: Diagramma delle classi – Search: IPC Adapter e Use Cases

SearchIpcAdapter :

- **Descrizione**: adapter inbound che registra su *IpcMain* di Electron i canali di comunicazione per il modulo di ricerca. Risolve tramite il container DI (tsyringe) le dipendenze necessarie e instrada le richieste provenienti dal renderer process verso i rispettivi Use Case. Non contiene logica di business.
- ***+ register(ipcMain: IpcMain)**: void*: metodo statico che registra tutti i canali IPC del modulo. Per ciascun canale, risolve dal container il corrispondente Use Case o adapter e ne invoca il metodo *execute()*, restituendo i risultati mappati in DTO di presentazione.
- **Canali registrati**:
  - *SEARCH_CLASSES*: ricerca le classi documentali per nome, delegando a *ISearchDocumentalClassUC* e mappando i risultati tramite *DocumentClassMapper.toSearc*
  - *SEARCH_PROCESSES*: ricerca i processi per UUID, delegando a *ISearchProcessUC* e mappando tramite *ProcessMapper.toSearchResult()*.
  - *SEARCH_DOCUMENTS*: ricerca i documenti tramite filtri strutturati. Valida il *SearchRequestDTO* con Zod, normalizza i path dei metadati tramite

MetadataKeyMapper.mapGroup(), costruisce un *SearchDocumentsQuery* e delega a *ISearchDocumentsUC*. I risultati vengono mappati tramite *DocumentMapper.toSea*

- *SEARCH_SEMANTIC*: ricerca semantica testuale, delegando a *ISearchSemanticUC* e mappando i risultati (con score di similarità) tramite *DocumentMapper.toSearchResult()*.
- *SEARCH_FULLTEXT*: ricerca full-text polimorfica che, in base al *SearchQueryType* della query (*PROCESS_ID*, *CLASS_NAME*, *FREE*), instrada la richiesta al Use Case appropriato.
- *SEARCH_GET_AI_STATE*: restituisce lo stato del motore AI, interrogando *IWordEmbedding.isInitialized()* e *IDocumentRepository.getIndexedDocumentsCount()*, e costruendo un oggetto con *status*, *progressPercentage* e *indexedDocuments*.
- *SEARCH_CUSTOM_METADATA_KEYS*: recupera le chiavi di metadati personalizzati, delegando a *IGetCustomMetadataKeysUC*.

7.2.2 Use Case di ricerca

SearchDocumentalClassUC :

- **Descrizione**: Use Case che implementa *ISearchDocumentalClassUC*, responsabile della ricerca delle classi documentali per nome. Delega interamente al repository senza applicare logica di business aggiuntiva.
- ***+ execute(name: string)***: *DocumentClass[]*: invoca *IDocumentClassRepository.searchDocume* e restituisce l'array di entità *DocumentClass* corrispondenti.

SearchProcessUC :

- **Descrizione**: Use Case che implementa *ISearchProcessUC*, responsabile della ricerca dei processi per UUID.
- ***+ execute(uuid: string)***: *Process[]*: invoca *IProcessRepository.searchProcesses(uuid)* e restituisce l'array di entità *Process* corrispondenti.

SearchDocumentsUC :

- **Descrizione**: Use Case che implementa *ISearchDocumentsUC*, responsabile della ricerca strutturata dei documenti tramite un albero di filtri logici. Riceve un *SearchDocumentsQuery* già costruito dall'adapter e lo passa al repository.
- ***+ execute(filters: SearchDocumentsQuery)***: *Document[]*: invoca *IDocumentRepository.searc* e restituisce l'array di entità *Document* corrispondenti ai criteri di filtro.

SearchSemanticUC :

- **Descrizione**: Use Case che implementa *ISearchSemanticUC*, responsabile della ricerca semantica dei documenti tramite similarità vettoriale. Coordina *IWordEmbedding* per la generazione del vettore di query e *IDocumentRepository* per la ricerca tra i vettori indicizzati.
- ***+ execute(query: string)***: *Promise<SemanticSearchMatch[]>*: converte la stringa di query in un vettore tramite *IWordEmbedding.generateEmbedding()*, quindi invoca *IDocumentRepository.searchDocumentSemantic()* e restituisce un array di *SemanticSearchMatch*, ciascuno contenente il *Document* e il relativo *score* di similarità.

GetCustomMetadataKeysUC :

- **Descrizione**: Use Case che implementa *IGetCustomMetadataKeysUC*, responsabile del recupero delle chiavi di metadati personalizzati presenti nel pacchetto.
- ***+ execute(dipId: number | null)***: *string[]*: invoca *IDocumentRepository.getDistinctCustomMe* e restituisce l'array di chiavi distinte. Se *dipId* è *null*, le chiavi vengono recuperate sull'intero database senza filtrare per pacchetto.

7.2.3 Modelli di dominio per la ricerca

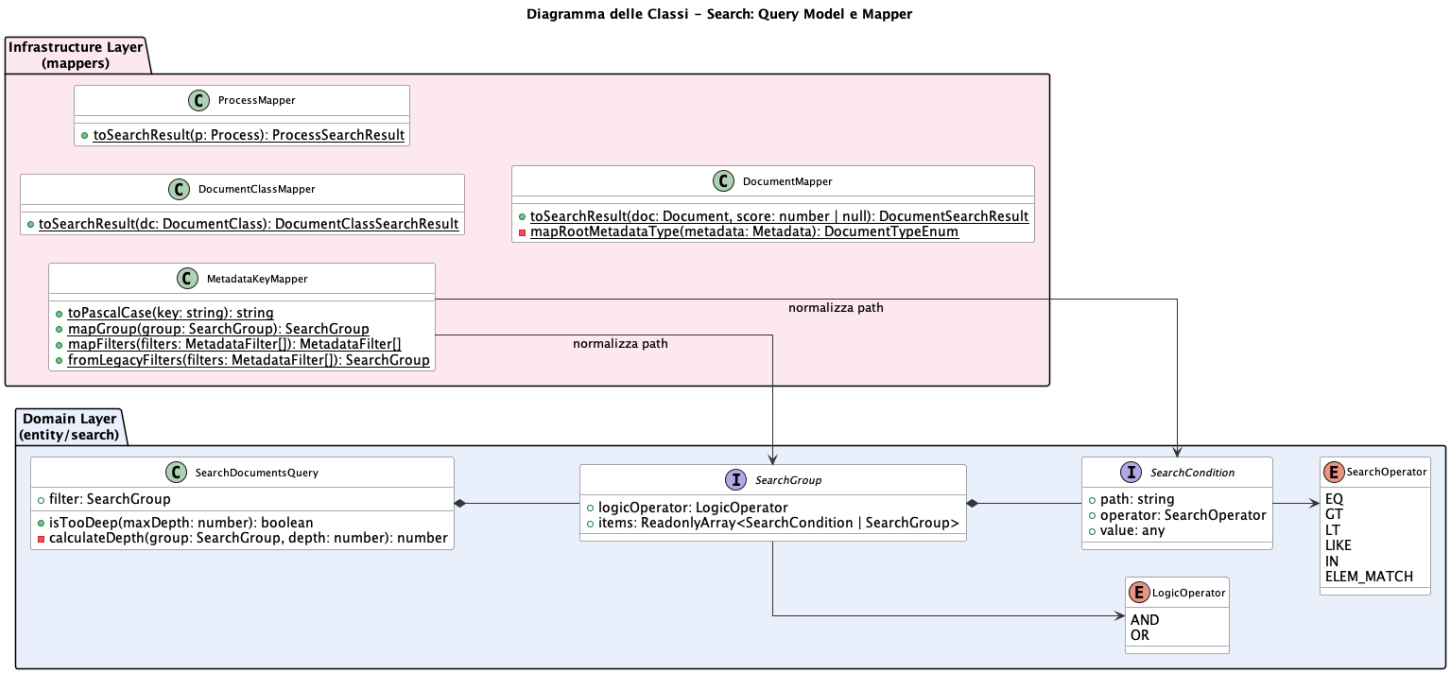

Figura 6: Diagramma delle classi – Search: Query Model e Mapper

SearchDocumentsQuery :

- **Descrizione**: oggetto query che incapsula l'albero di filtri logici da applicare alla ricerca dei documenti. Viene costruito dall'adapter a partire dal *SearchRequestDTO* validato e passato allo Use Case e poi al DAO.
- ***+ filter: SearchGroup***: radice dell'albero di filtri, contenente l'operatore logico e gli elementi figli.
- ***+ isTooDeep(maxDepth: number)**: boolean*: metodo di utilità che calcola ricorsivamente la profondità dell'albero di filtri e restituisce *true* se supera il limite *maxDepth* (default 5), per prevenire query eccessivamente annidate che potrebbero degradare le performance del database.

SearchGroup :

- **Descrizione**: interfaccia che rappresenta un nodo dell'albero di filtri, contenente un operatore logico e un array di elementi figli. Ogni elemento può essere a sua volta una *SearchGroup* (nodo) o una *SearchCondition* (foglia), realizzando una struttura ricorsiva che implementa il **Composite pattern**.
- ***logicOperator: LogicOperator***: operatore logico (*AND* o *OR*) che combina gli elementi figli del gruppo.
- ***items: ReadonlyArray<SearchCondition | SearchGroup>***: array degli elementi figli del gruppo.

SearchCondition :

- **Descrizione**: interfaccia che rappresenta una foglia dell'albero di filtri, ovvero una singola condizione atomica da applicare a un campo del documento.
- ***path: string***: percorso dot-notation che identifica il campo del documento su cui applicare la condizione (es. *NomeDelDocumento* o *CustomMetadata.codice*).
- ***operator: SearchOperator***: operatore di confronto da applicare. I valori supportati sono *EQ* (uguaglianza), *GT* (maggiore), *LT* (minore), *LIKE* (sottostringa), *IN* (appartenenza a lista) e *ELEM_MATCH* (corrispondenza su elementi di un array annidato).
- ***value: any***: valore con cui confrontare il campo. Per *IN* deve essere un array; per *ELEM_MATCH* deve essere un *SearchGroup* annidato.

SearchRequestDTO e SearchRequestSchema :

- **Descrizione**: DTO e schema di validazione Zod che rappresentano la richiesta di ricerca strutturata proveniente dal frontend. *SearchRequestSchema* valida ricorsivamente l'albero di filtri, garantendo la correttezza strutturale prima che la richiesta raggiunga il dominio. Il path di ogni *SearchCondition* è validato tramite una regex che consente solo la dot-notation alfanumerica.

- ***filter: SearchGroupDTO***: radice dell'albero di filtri nella rappresentazione DTO, strutturalmente equivalente a *SearchGroup* ma non tipizzata con le interfacce di dominio.

SearchQuery :

- **Descrizione**: interfaccia che rappresenta una query di ricerca full-text o semantica proveniente dal frontend.
- ***text: string***: testo della query inserito dall'utente.
- ***type: SearchQueryType***: tipo della query (*PROCESS_ID*, *CLASS_NAME*, *FREE*), utilizzato da *SearchIpcAdapter* per instradare la richiesta al Use Case appropriato nel canale *SEARCH_FULLTEXT*.
- ***useSemanticSearch: boolean***: flag che indica se utilizzare la ricerca semantica.

7.2.4 Mapper di ricerca

I mapper di ricerca trasformano le entità di dominio nei DTO di presentazione restituiti al frontend. Sono metodi statici collocati nei rispettivi mapper di persistenza per coerenza architetturale. Il diagramma di riferimento è riportato in figura 6.

DocumentClassMapper.toSearchResult() :

- **Descrizione**: trasforma un'entità *DocumentClass* in un *DocumentClassSearchResult*, verificando che l'entità sia già persistita (e quindi abbia un *id* valido). Il campo *type* è fisso a *"CLASSE"*.
- **Output**: *DocumentClassSearchResult* con campi *id*, *uuid*, *integrityStatus*, *name* e *type*.

ProcessMapper.toSearchResult() :

- **Descrizione**: trasforma un'entità *Process* in un *ProcessSearchResult*, verificando che l'entità sia già persistita. Il campo *type* è fisso a *"PROCESSO"*.
- **Output**: *ProcessSearchResult* con campi *id*, *uuid*, *integrityStatus* e *type*.

DocumentMapper.toSearchResult() :

- **Descrizione**: trasforma un'entità *Document* e uno score opzionale in un *DocumentSearchResult*. Estrae il nome del documento cercando prima il nodo *NomeDelDocumento* e poi *nome* nell'albero dei metadati. Determina il tipo del documento (*DocumentTypeEnum*) mappando il nome del nodo radice dei metadati tramite *mapRootMetadataType()*, che normalizza il nome in *UPPER_SNAKE_CASE* e lo confronta con i valori canonici attesi.
- **Output**: *DocumentSearchResult* con campi *id*, *uuid*, *integrityStatus*, *name*, *type* e *score*.

MetadataKeyMapper :

- **Descrizione**: utility class che normalizza i path dei metadati provenienti dal frontend in PascalCase, garantendo coerenza con la struttura interna del JSON dei metadati dei documenti.
- ***+ toPascalCase(key: string)**: string*: converte una chiave in PascalCase, gestendo separatori come _ e -, e supportando path dot-notation trasformando ogni segmento separatamente.
- ***+ mapGroup(group: SearchGroup)**: SearchGroup*: applica ricorsivamente *toPascalCase()* al campo *path* di ogni *SearchCondition* nell'albero, inclusi i valori di condizioni *ELEM_MATCH* che contengono un *SearchGroup* annidato. Restituisce un nuovo albero con tutti i path normalizzati.
- ***+ mapFilters(filters: MetadataFilter[])**: MetadataFilter[]*: variante per array piatti di filtri chiave-valore, normalizza il campo *key* di ciascun filtro.
- ***+ fromLegacyFilters(filters: MetadataFilter[])**: SearchGroup*: metodo di compatibilità che converte un array piatto di filtri legacy in un *SearchGroup* con operatore *AND*, filtrando i valori nulli o vuoti e impostando l'operatore *EQ* per ogni condizione.

7.2.5 Repository e DAO di ricerca

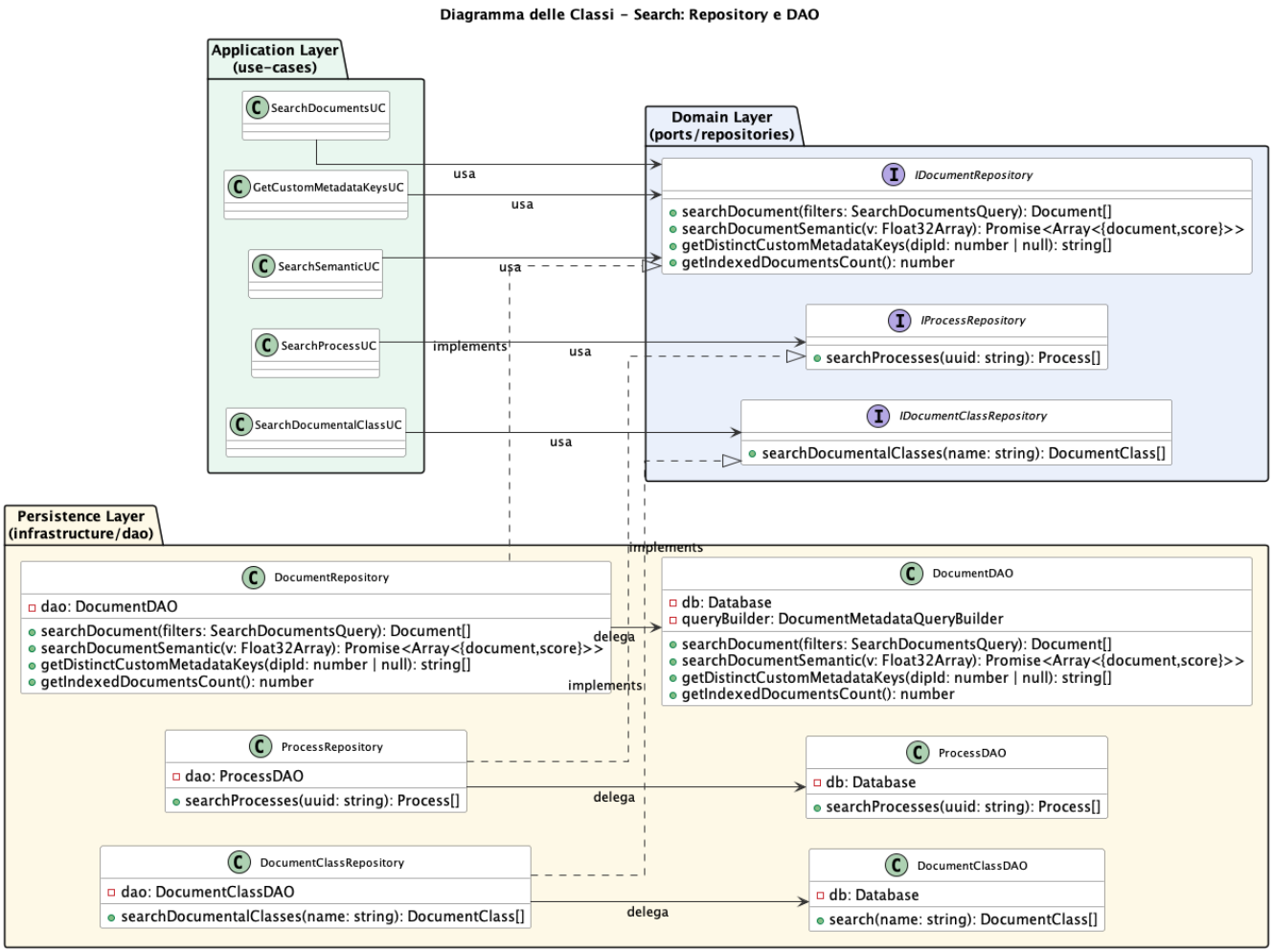

Figura 7: Diagramma delle classi – Search: Repository e DAO

I repository implementano il pattern Repository delegando le operazioni di persistenza ai rispettivi DAO, che interagiscono direttamente con SQLite tramite *better-sqlite3*. Di seguito sono documentati i metodi specifici della ricerca.

DocumentClassRepository e DocumentClassDAO :

- **Descrizione**: *DocumentClassRepository* implementa *IDocumentClassRepository* delegando a *DocumentClassDAO*, che esegue le query SQL sul database SQLite. Per la ricerca, *DocumentClassRepository* espone *searchDocumentalClasses(name)*, che internamente invoca *DocumentClassDAO.search(name)*.
- ***search(name: string)***: *DocumentClass[]*: esegue una query SQL con *WHERE name LIKE %nam* sulla tabella *document_class* e mappa le righe risultanti in entità *DocumentClass* tramite *DocumentClassMapper.fromPersistence()*.

ProcessRepository e ProcessDAO :

- **Descrizione**: *ProcessRepository* implementa *IProcessRepository* delegando a *ProcessDAO*. Per la ricerca, espone *searchProcesses(uuid)*, che internamente invoca *ProcessDAO.searchProcesses(uuid)*.
- ***searchProcesses(uuid: string)**: Process[]*: esegue una query SQL con *WHERE uuid LIKE %uui* sulla tabella *process*, ordinando per *id*, e mappa le righe in entità *Process*.

DocumentRepository e DocumentDAO :

- **Descrizione**: *DocumentRepository* implementa *IDocumentRepository* delegando a *DocumentDAO*. Espone due metodi di ricerca principali: *searchDocument()* per la ricerca strutturata e *searchDocumentSemantic()* per la ricerca semantica.
- ***searchDocument(filters: SearchDocumentsQuery)**: Document[]*: delega a *DocumentDAO.sea* che utilizza *DocumentMetadataQueryBuilder* per tradurre il *SearchDocumentsQuery* in una query SQL parametrizzata con *json_extract()* su SQLite, ed esegue la query restituendo le entità *Document* mappate tramite *DocumentMapper.fromJsonPersistence()*.
- ***searchDocumentSemantic(queryVector: Float32Array)**: Promise<Array<{document, score}>* delega a *DocumentDAO.searchDocumentSemantic()*, che carica tutti i vettori dalla tabella *document_vector*, calcola la cosine similarity tra ciascuno e il *queryVector*, ordina i risultati per score decrescente, seleziona i primi 10 e restituisce le coppie *{document, score}* filtrate dai valori nulli. Restituisce un array vuoto se la tabella non esiste o il vettore di query è privo di dimensioni.
- ***getDistinctCustomMetadataKeys(dipId: number | null)**: string[]*: recupera le chiavi di metadati personalizzati distinte, combinando due strategie: una query SQL che cerca nei nodi figli dei contenitori *CustomMetadata* e *ArchimemoData* nelle tabelle *document_metadata* e *process_metadata*, e una visita ricorsiva dei blob JSON nella colonna *metadata* della tabella *document*. I risultati delle due strategie vengono deduplicati e ordinati alfabeticamente.
- ***getIndexedDocumentsCount()**: number*: restituisce il numero di documenti con vettore indicizzato nella tabella *document_vector*, o 0 se la tabella non esiste.

7.2.6 DocumentMetadataQueryBuilder

DocumentMetadataQueryBuilder :

- **Descrizione**: componente responsabile della traduzione di un *SearchDocumentsQuery* in una clausola SQL parametrizzata compatibile con SQLite. Utilizza le funzioni *json_extract()* e *json_each()* di SQLite per navigare la struttura JSON dei metadati dei documenti. Viene istanziato direttamente da *DocumentDAO*.

- ***+ build(filters: SearchDocumentsQuery)**: {sql: string, params: unknown[]}*: punto di ingresso pubblico. Verifica che il filtro non sia vuoto, quindi delega a *buildQueryFromGroup()* sulla radice del *SearchGroup*. Restituisce la clausola SQL e i parametri da passare alla query preparata.
- ***- buildQueryFromGroup(group: SearchGroup, jsonSource: string)**: {sql, params}*: costruisce ricorsivamente la clausola SQL per un *SearchGroup*. Per ogni elemento del gruppo, distingue tra nodi (*SearchGroup* annidato, gestito ricorsivamente) e foglie (*SearchCondition*, delegate a *buildCondition()*). Le clausole risultanti vengono unite con *AND* o *OR* in base al *logicOperator* del gruppo.
- ***- buildCondition(condition: SearchCondition, jsonSource: string)**: {sql, params}*: traduce una singola *SearchCondition* in SQL. Per l'operatore *ELEM_MATCH*, genera una sottoquery con *EXISTS (SELECT 1 FROM json_each(...))*, permettendo il filtraggio su array JSON annidati. Per tutti gli altri operatori, delega a *buildScalarCondition()* utilizzando *json_extract()* per accedere al campo.
- ***- buildScalarCondition(jsonExtract, operator, value)**: {sql, params}*: genera la clausola SQL per operatori scalari. Per *EQ* su valori numerici, applica un doppio confronto per gestire la coerenza tra tipi numerici e stringhe nel JSON. Per *LIKE* utilizza *COLLATE NOCASE*. Per *IN* filtra i valori nulli o vuoti prima di costruire i placeholder.
- ***- toJsonPath(path: string)**: string*: converte un path dot-notation (es. *CustomMetadata.codice*) nella sintassi JSONPath di SQLite (es. *$.CustomMetadata.codice*). Valida ogni segmento con una regex e solleva un'eccezione in caso di segmenti non validi.